



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

عنوان:

آزمایش پنجم: طراحی ضرب کننده

نام درس

آزمایشگاه طراحی سیستم های دیجیتال

نیم سال ۱۴۰۴

استاد

دکتر انصاری

گروه ۸

امیرشایان سلیمانی نیا - ۴۰۳۱۰۶۱۰۸

حسن پناهی - ۴۰۳۱۰۵۸۳۳

امین استادکاظمی - ۴۰۳۱۷۰۸۹۳

فهرست مطالب

۱	مقدمه و هدف آزمایش	۲
۲	شرح آزمایش	۲
۳	توضیح ساختار مدار	۳
۳	۱.۳ ماژول Datapath	۳
۳	۲.۳ ماژول Controller	۳
۴	۳.۳ ماژول اصلی	۴
۴	کدهای وریلگ مدار و تست بنچ	۵
۵	۱.۴ کد اصلی مدار (booth_mult)	۵
۷	۲.۴ کد Testbench	۷
۵	تست مدار و waveform	۹
۶	نتیجه گیری	۱۰

۱ مقدمه و هدف آزمایش

هدف از انجام این آزمایش، طراحی، توصیف سخت‌افزاری و شبیه‌سازی یک ضرب‌کننده با استفاده از الگوریتم Booth است. در این سیستم واحدهای Datapath و Controller به‌صورت جداگانه پیاده‌سازی شده و در کنار هم عملیات ضرب را مدیریت می‌کنند.

۲ شرح آزمایش

در این آزمایش یک ضرب‌کننده هشت‌بیتی مدل‌سازی شده است. قوانین و ویژگی‌های کنترلی این سیستم به شرح زیر است:

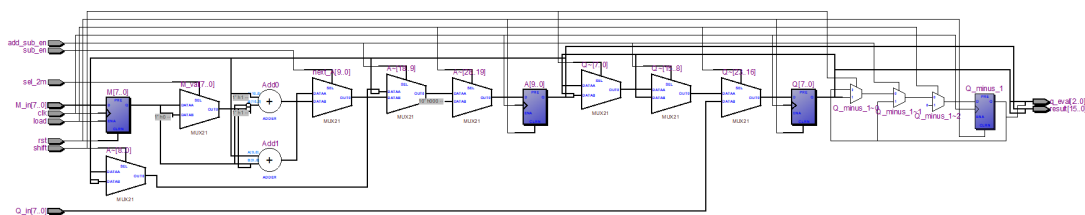
- **شیفت چندگانه:** واحد شیفت‌دهنده توانایی شیفت بیش از یک بیت در هر پالس ساعت را دارد. این ویژگی با استفاده از الگوریتم Radix-4 محقق می‌شود تا در هر مرحله دو بیت بررسی شده و سرعت عملیات نسبت به ضرب‌کننده‌های ساده‌ی Shift & Add افزایش یابد.
- **جلوگیری از خطای سرریز:** در ضرب مقادیر مرزی مانند ۱۲۸- به دلیل نیاز به تولید مقدار $-2M$ ، احتمال وقوع سرریز در داده‌ها وجود دارد. برای رفع این مشکل، طول رجیسترها از پیش محاسبه شده و انباشتگر سیستم به‌صورت ده‌بیتی طراحی شده است تا محاسبات بدون خطا انجام شوند.
- **مدیریت جریان داده:** کنترلر با بررسی سه بیت ارزیابی در هر مرحله، فرمان‌های لازم جهت جمع یا تفریق با M و $2M$ را تولید کرده و سپس دستور شیفت را به مسیر داده صادر می‌کند.

۳ توضیح ساختار مدار

طراحی این سیستم شامل دو بخش اصلی و یک ماژول متصل کننده است:

۱.۳ Datapath ماژول

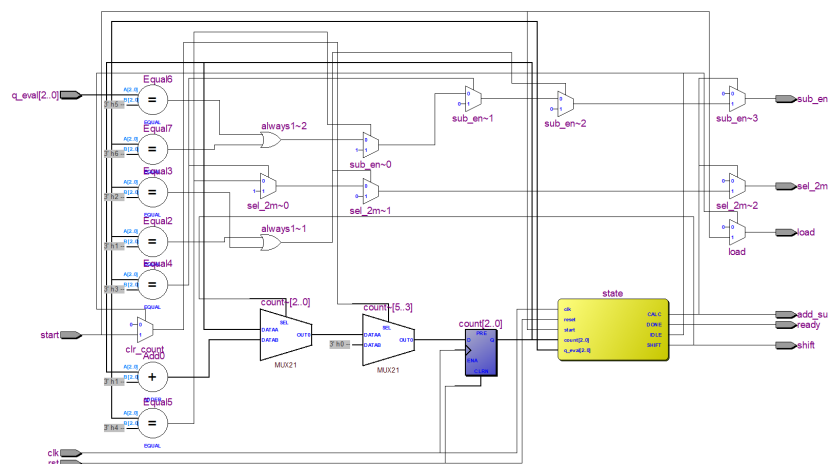
این ماژول ارتباط بین رجیسترها، مالتی پلکسرها و جمع کننده ها را مدیریت می کند. در معماری این بخش، از سه رجیستر اصلی استفاده شده است؛ رجیسترهای M و Q برای نگهداری مقسوم علیه و مضرب، و یک رجیستر ده بیتی به نام A که وظیفه انباشت حاصل جمع های میانی را بر عهده دارد. برای پیاده سازی منطق الگوریتم، مقدار $2M$ با یک شیفت سخت افزاری ایجاد شده و مالتی پلکسرها بر اساس سیگنال های کنترلی، داده ی مورد نیاز را برای جمع یا تفریق به انباشتگر هدایت می کنند.



شکل ۱: نمای RTL مدار داخلی مسیر داده

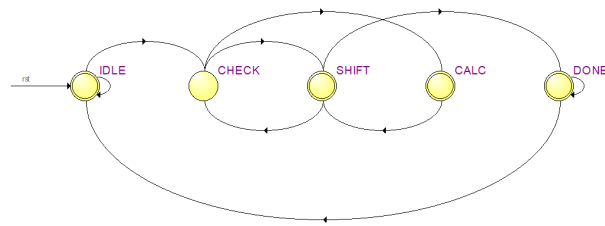
۲.۳ Controller ماژول

این ماژول به صورت یک FSM پنج حالت طراحی شده است. مدار در وضعیت IDLE منتظر سیگنال start می ماند. با شروع کار، مدار وارد وضعیت CHECK شده تا سه بیت q_eval را ارزیابی کند. در صورتی که این سه بیت تماماً یک یا تماماً صفر باشند، نیازی به محاسبات نیست و مدار وارد وضعیت SHIFT می شود. در غیر این صورت، در وضعیت CALC بر اساس ارزش بیت ها، مقادیر M یا $2M$ جمع و تفریق شده و سپس به مرحله شیفت می رود.



شکل ۲: نمای RTL گیت های داخلی واحد کنترل

روند اجرای الگوریتم و جابجایی بین حالت‌های ماشین وضعیت، در نمودار زیر قابل مشاهده است.



شکل ۳: نمودار حالت‌های ماشین

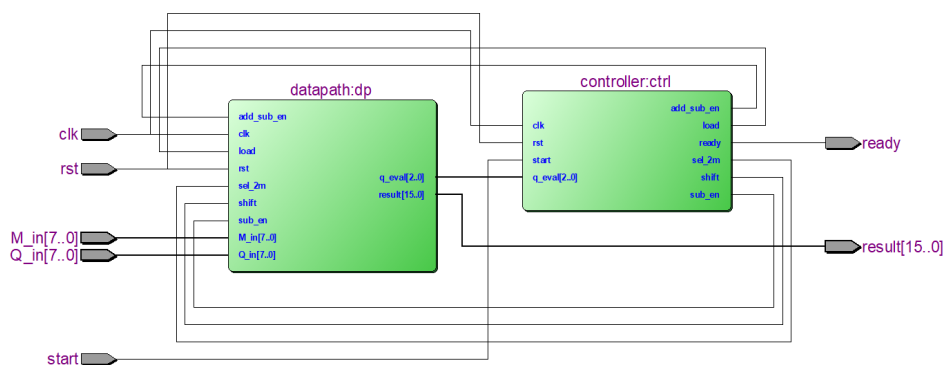
همچنین منطق ترکیب این حالت‌ها و شروطی که باعث انتقال وضعیت کنترلر در هر کلاک پالس می‌شوند، در جدول زیر استخراج شده است.

	Source State	Destination State	Condition
1	CALC	SHIFT	$(!q_eval[0]) \cdot (!q_eval[1]) \cdot (!q_eval[2]) + (q_eval[0]) \cdot (q_eval[1]) \cdot (q_eval[2])$
2	CHECK	SHIFT	$(!q_eval[0]) \cdot (!q_eval[1]) \cdot (q_eval[2]) + (q_eval[0]) \cdot (q_eval[1]) \cdot (q_eval[2])$
3	CHECK	CALC	$(!q_eval[0]) \cdot (!q_eval[1]) \cdot (q_eval[2]) + (q_eval[0]) \cdot (q_eval[1]) \cdot (q_eval[2])$
4	DONE	IDLE	$(!start)$
5	DONE	DONE	$(start)$
6	IDLE	IDLE	$(!start)$
7	IDLE	CHECK	$(start)$
8	SHIFT	DONE	$(count[0]) \cdot (count[1]) \cdot (!count[2])$
9	SHIFT	CHECK	$(!count[0]) + (count[0]) \cdot (!count[1]) + (count[0]) \cdot (count[1]) \cdot (count[2])$

شکل ۴: جدول گذار حالت‌ها و منطق انتقال

۳.۳ ماژول اصلی

این ماژول به عنوان بدنه اصلی، دو بلوک قبلی را در کنار یکدیگر قرار می‌دهد. تمامی سیگنال‌های کنترلی اعم از فعال‌ساز جمع، تفریق و شیفت از کنترلر به مسیر داده وارد شده و وضعیت پردازش نیز از مسیر داده به کنترلر فیدبک داده می‌شود.



شکل ۵: شماتیک یکپارچه و اتصال واحد کنترلر به مسیر داده

۴ کدهای وریلاگ مدار و تست‌بنچ

در این بخش، کدهای مربوط به توصیف سخت‌افزاری مدار و محیط شبیه‌سازی آن آورده شده است.

۱.۴ کد اصلی مدار (booth_mult)

این کد شامل سه بخش است. ابتدا ماژول datapath تعریف شده که وظیفه نگهداری داده‌ها و انجام عملیات ریاضی را بر عهده دارد. در ادامه ماژول controller قرار دارد که به عنوان هسته مرکزی، فرمان‌های کنترلی را صادر می‌کند. در نهایت، ماژول booth_mult این دو بلوک را فراخوانی کرده و پورت‌های ورودی و خروجی کل سیستم را یکپارچه می‌کند.

```
module datapath (
    input clk, rst,
    input [7:0] M_in, Q_in,
    input load, shift, add_sub_en, sub_en, sel_2m,
    output [2:0] q_eval,
    output [15:0] result
);
    reg [9:0] A;
    reg [7:0] M, Q;
    reg Q_minus_1;

    wire [9:0] M_ext = {{2{M[7]}}, M};
    wire [9:0] M_val = sel_2m ? {{1{M[7]}}, M, 1'b0} : M_ext;
    wire [9:0] next_A = sub_en ? (A - M_val) : (A + M_val);

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            A <= 0;
            M <= 0;
            Q <= 0;
            Q_minus_1 <= 0;
        end else if (load) begin
            A <= 0;
            M <= M_in;
            Q <= Q_in;
            Q_minus_1 <= 0;
        end else if (add_sub_en) begin
            A <= next_A;
        end else if (shift) begin
            A <= {A[9], A[9], A[9:2]};
            Q <= {A[1:0], Q[7:2]};
            Q_minus_1 <= Q[1];
        end
    end

    assign q_eval = {Q[1:0], Q_minus_1};
    assign result = {A[7:0], Q};
endmodule

module controller (
    input clk, rst, start,
    input [2:0] q_eval,
    output reg load, add_sub_en, sub_en, sel_2m, shift, ready
);
    reg [2:0] state, next_state;
    reg [2:0] count;
    reg inc_count, clr_count;
```

```

localparam IDLE = 3'd0, CHECK = 3'd1, CALC = 3'd2, SHIFT = 3'd3, DONE = 3'd4;

always @(posedge clk or posedge rst) begin
    if (rst) begin
        state <= IDLE;
        count <= 0;
    end else begin
        state <= next_state;
        if (clr_count) count <= 0;
        else if (inc_count) count <= count + 1;
    end
end

always @(*) begin
    next_state = state;
    load = 0; add_sub_en = 0; sub_en = 0; sel_2m = 0; shift = 0; ready = 0;
    inc_count = 0; clr_count = 0;

    case (state)
        IDLE: begin
            if (start) begin
                load = 1;
                clr_count = 1;
                next_state = CHECK;
            end
        end
        CHECK: begin
            if (q_eval == 3'b000 || q_eval == 3'b111) begin
                next_state = SHIFT;
            end else begin
                next_state = CALC;
            end
        end
        CALC: begin
            add_sub_en = 1;
            if (q_eval == 3'b001 || q_eval == 3'b010) begin
                sub_en = 0; sel_2m = 0;
            end else if (q_eval == 3'b011) begin
                sub_en = 0; sel_2m = 1;
            end else if (q_eval == 3'b100) begin
                sub_en = 1; sel_2m = 1;
            end else if (q_eval == 3'b101 || q_eval == 3'b110) begin
                sub_en = 1; sel_2m = 0;
            end
            next_state = SHIFT;
        end
        SHIFT: begin
            shift = 1;
            inc_count = 1;
            if (count == 3) next_state = DONE;
            else next_state = CHECK;
        end
        DONE: begin
            ready = 1;
            if (!start) next_state = IDLE;
        end
    endcase
end
endmodule

module booth_mult (
    input clk, rst, start,

```

```

    input [7:0] M_in, Q_in,
    output [15:0] result,
    output ready
);
wire load, shift, add_sub_en, sub_en, sel_2m;
wire [2:0] q_eval;

datapath dp (
    .clk(clk),
    .rst(rst),
    .M_in(M_in),
    .Q_in(Q_in),
    .load(load),
    .shift(shift),
    .add_sub_en(add_sub_en),
    .sub_en(sub_en),
    .sel_2m(sel_2m),
    .q_eval(q_eval),
    .result(result)
);

controller ctrl (
    .clk(clk),
    .rst(rst),
    .start(start),
    .q_eval(q_eval),
    .load(load),
    .add_sub_en(add_sub_en),
    .sub_en(sub_en),
    .sel_2m(sel_2m),
    .shift(shift),
    .ready(ready)
);
endmodule

```

۲.۴ کد Testbench

این کد برای ارزیابی عملکرد ضرب‌کننده نوشته شده است. در این تست‌بنچ با استفاده از یک تابع (task)، مقادیر مختلفی اعم از اعداد مثبت، منفی، صفر و داده‌های مرزی به مدار اعمال می‌شود. تست‌بنچ به صورت خودکار خروجی مدار را با جواب واقعی ضرب مقایسه کرده و در صورت صحت عملکرد، PASS را در ترمینال نمایش می‌دهد.

```

`timescale 1ns/1ps

module tb_booth_mult();
    reg clk, rst, start;
    reg signed [7:0] M, Q;
    wire signed [15:0] result;
    wire ready;

    reg signed [15:0] expected;

    booth_mult uut (
        .clk(clk),
        .rst(rst),
        .start(start),
        .M_in(M),
        .Q_in(Q),
        .result(result),
        .ready(ready)
    );
endmodule

```

```

);

always #5 clk = ~clk;

task test_multiply;
    input signed [7:0] m_val;
    input signed [7:0] q_val;
    begin
        @(negedge clk);
        M = m_val;
        Q = q_val;
        expected = M * Q;
        start = 1;

        @(negedge clk);
        start = 0;

        wait(ready == 1);

        if (result != expected)
            $display("ERROR: Time=%0t | %d x %d = %d (Expected: %d)", $time, M, Q, result, expected);
        else
            $display("PASS: Time=%0t | %d x %d = %d", $time, M, Q, result);

        wait(ready == 0);
    end
endtask

initial begin
    clk = 0; rst = 1; start = 0; M = 0; Q = 0;
    #15 rst = 0;

    test_multiply(8'd7, 8'd3);
    test_multiply(8'sd5, -8'sd4);
    test_multiply(-8'sd6, 8'sd7);
    test_multiply(-8'sd8, -8'sd3);

    test_multiply(8'd0, 8'd12);
    test_multiply(-8'sd15, 8'd0);
    test_multiply(8'd0, 8'd0);

    test_multiply(8'sd127, 8'sd127);
    test_multiply(-8'sd128, -8'sd128);
    test_multiply(-8'sd128, 8'sd127);
    test_multiply(8'sd127, -8'sd128);

    #20 $stop;
end
endmodule

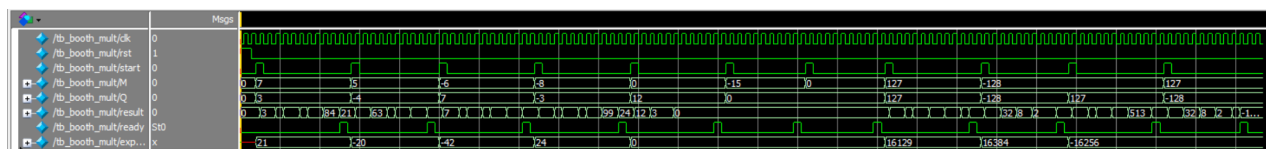
```

۵ تست مدار و waveform

جهت بررسی صحت عملکرد مدار یک Testbench نوشته شد و در نرم افزار ModelSim مورد شبیه سازی قرار گرفت. لاگ خروجی و شکل موج این شبیه سازی در ادامه قابل مشاهده است:

```
VSIM 22> run -all
# PASS: Time=125000 |      7 x      3 =      21
# PASS: Time=235000 |      5 x     -4 =     -20
# PASS: Time=355000 |     -6 x      7 =     -42
# PASS: Time=475000 |     -8 x     -3 =      24
# PASS: Time=595000 |      0 x     12 =      0
# PASS: Time=695000 |    -15 x      0 =      0
# PASS: Time=795000 |      0 x      0 =      0
# PASS: Time=915000 |    127 x   127 =   16129
# PASS: Time=1025000 |  -128 x  -128 =   16384
# PASS: Time=1145000 |  -128 x   127 =  -16256
# PASS: Time=1255000 |    127 x  -128 =  -16256
```

شکل ۶: خروجی ترمینال و بررسی خودکار نتایج مدار



شکل ۷: شکل موج های شبیه سازی مدار در نرم افزار ModelSim

در این شکل موج، سناریوهای زیر به ترتیب زمان اجرا و تحلیل شده اند:

۱. مقداردهی اولیه: در ابتدا با اعمال سیگنال صفر و یک به پایه rst، مقادیر تمامی رجیسترها ریست شده و سیستم در حالت IDLE آماده دریافت فرمان می شود.
۲. ضرب اعداد مثبت: دو عدد ۷ و ۳ وارد سیستم شده و پالس start اعمال می شود. مدار طی چهار چرخه ی کلاک عملیات ارزیابی و شیفت را انجام داده و با یک شدن سیگنال ready، مقدار صحیح ۲۱ را تثبیت می کند.
۳. مدیریت علامت ها: در پالس های بعدی اعداد با علامت های متفاوت (مانند ضرب ۵ در ۴-) وارد

سیستم می‌شوند. کنترلر با ارزیابی صحیح بیت‌ها، نتیجه را با علامت منفی به درستی در result قرار می‌دهد.

۴. **حالت ضرب در صفر:** با قرار گرفتن یکی از ورودی‌ها روی صفر، ماشین حالت بلافاصله شرایط تمام صفر را تشخیص داده و بدون انجام عملیات جمع یا تفریق اضافه، تنها شیفت را اعمال می‌کند که در نتیجه عدد صفر به دست می‌آید.

۵. **تست ظرفیت اکستریم:** همان‌طور که در انتهای شکل موج مشاهده می‌شود، مدار با تشخیص صحیح ضرب 2M- و به خاطر طراحی رجیستر ده‌بیتی، بدون هیچ‌گونه خطایی، عدد ۱۶۳۸۴ را در خروجی تولید می‌کند.

۶ نتیجه‌گیری

در این آزمایش، یک ضرب‌کننده هشت‌بیتی با استفاده از الگوریتم Booth در زبان Verilog طراحی و پیاده‌سازی شد. با تفکیک طراحی به دو ماژول مجزای Datapath و Controller، منطق مدار به شکل ساختاریافته‌تری مدل‌سازی گردید. پس از نوشتن توصیف سخت‌افزاری، مدار را در نرم‌افزار Quartus سنتز کرده و شماتیک RTL مربوط به بلوک‌ها و ماشین وضعیت را استخراج کردیم. در نهایت، با استفاده از نرم‌افزار ModelSim و نوشتن یک Testbench، عملکرد مدار شبیه‌سازی شد که بررسی شکل موج‌های خروجی نشان داد مدار در شرایط مختلف محاسباتی عملکرد صحیحی دارد.